



4-for

1. 简介



本节介绍 `for` 循环语句。

学习目标：学完本节后，你应该能够：

- 掌握 `for` 语句的语法与执行流程（init-clause、cond-expression、iteration-expression）；
- 对比 `while` 与 `for`，理解 `for` 把"初始化、条件、增量"集中到一处的好处；
- 在循环中配合 `break` 提前退出、用 `bool` 变量记录状态；

本节重点：

- `for` 语句的语法与语义；
- `for` 与 `while` 的对比；
- 用 `for` 遍历数组与双重循环。

本节难点：

- 循环不变式（选择排序）；
- 双重循环的执行过程。

2. `min-array`

求一组整数之间的最小值。



根据公式 $\min(23, 56, 19, 11, 78) = \min(\min(\min(\min(23, 56), 19), 11), 78)$ ，计算 5 个元素的最小值，可以通过 4 次迭代地计算两个元素的最小值完成。“迭代地”（iteratively）做某件事是使用循环结构的根本原因，“某件事”就构成了循环的主体部分，即循环体（loop body），而这里的 4 次则是在控制循环次数，其背后隐藏的是循环得以持续进行的条件（loop condition）。

2.1 min-array-while

先用 `while` 循环实现上述迭代过程：

```
min-array-while.cpp

1  // File: min-array-while.cpp
2
3  #include <iostream>
4
5  int main() {
6      constexpr int NUM{5};
7      int numbers[NUM]{23, 56, 19, 11, 78};
8
9      int min{numbers[0]};
10     int i{1};
11     while (i < NUM) {
12         if (numbers[i] < min) {
13             min = numbers[i];
14         }
15         i++;
16     }
17
18     std::cout << "min = " << min << '\n';
19
20     return 0;
21 }
```

```
预期输出 = 11
```

用 `while` 写循环时，循环的三件事被分散在三处：在循环之前声明并初始化计数器 `int i{1}`；在 `while` 后的括号里写循环条件 `i < NUM`；在循环体末尾写计数器增量 `i++`。

能否把这三部分聚拢到一起，一眼看清循环的"初始化、条件、增量"？这正是 `for` 循环要解决的问题。

2.2 min-array-for

`min-array-for` 是 `min-array-while` 的 `for` 循环实现版本。

min-array-for.cpp

```
1 // File: min-array-for.cpp
2
3 #include <iostream>
4
5 int main() {
6     constexpr int NUM{5};
7     int numbers[NUM]{23, 56, 19, 11, 78};
8
9     int min{numbers[0]};
10    for (int i{1}; i < NUM; i++) {
11        if (numbers[i] < min) {
12            min = numbers[i];
13        }
14    }
15
16    std::cout << "min = " << min << '\n';
17
18    return 0;
19 }
```

预期输出

```
1 min = 11
```

2.2.1 for 语句的语法

`for@cppreference` 语句的语法 (syntax) 结构如下：

```
for (init-clause; cond-expression; iteration-expression) loop-statement
```

- `for` 是关键字 (keyword) ；
- `init-clause`：初始分句，可以是声明语句，也可以是表达式；
- `cond-expression`：表示循环条件的表达式；
- `iteration-expression`：用以迭代的表达式；
- `loop-statement`：循环体，通常放在 `{ }` 中。

`init-clause`、`cond-expression`、`iteration-expression` 三部分均是可选的，但是两个分号 `;` 是必要的。如果 `cond-expression` 为空，则根据 C++ 标准，相当于放置了一个非零整数，也就是一个永真的表达式，因此可以用来实现无限循环。

2.2.2 `for` 语句的语义

为了解释 `for` 语句的语义 (semantics, 即 `for` 语句是如何运行的)，我们如下标注 `for` 语句的各个部分，其中 (5) 表示 `for` 语句之后的第一条语句。

代码块

```
1   for (int i{1}; // (1) init-clause
2       i < NUM; // (2) cond-expression
3       i++; // (3) iteration-expression
4       { // (4) loop-statement
5       if (numbers[i] < min) {
6           min = numbers[i];
7       }
8   }
9   // (5) outside of the for loop
```

`for` 语句各个部分的执行顺序如下 (如图所示)：

首先，执行 (1) `init-clause`，进行准备工作。该部分通常是声明一些变量并进行初始化。该部分仅执行一次。

1. 然后，判断 (2) `cond-expression` 是否成立。
2. 如果 (2) `cond-expression` 成立，则执行 (4) `loop-statement`；否则，结束循环，开始执行 (5) 以及后续语句。
3. (4) `loop-statement` 执行结束后，转而执行 (3) `iteration-expression`，然后跳转到第 2 步。

综上所述，(1) `init-clause` 进行初始化，而 (2) `cond-expression`、(4) `loop-statement`、(3) `iteration-expression` 构成了一个循环。

2.2.3 `for` 语句运行过程演示

下表演示了 `for` 语句的运行过程：

迭代次数	i	条件 i < 5	numbers[i]	min	新 min	新 i
0	1	成立	56	23	23	2
1	2	成立	19	23	19	3
2	3	成立	11	19	11	4
3	4	成立	78	11	11	5
4	5	不成立	-	-	-	-

2.2.4 i < NUM 与数组越界访问

注意循环的条件是 `i < NUM`，而不是 `i <= NUM`。后者会造成数组访问越界（access out of bounds），而这也是一种未定义行为（undefined behavior, UB）！

2.3 min-array-input

很多时候，数组元素是由用户输入的。如果事先已知用户输入的元素个数，则可以在循环中使用 `std::cin` 依次读取用户的输入。

```
min-array-input-known-size.cpp

1  // File: min-array-input-known-size.cpp
2
3  #include <iostream>
4
5  int main() {
6      constexpr int NUM{5};
7      int numbers[NUM]{};
8
9      for (int i{0}; i < NUM; i++) {
10         std::cin >> numbers[i];
11     }
12
13     int min{numbers[0]};
14     for (int i{1}; i < NUM; i++) {
15         if (numbers[i] < min) {
16             min = numbers[i];
17         }
18     }
19
20     std::cout << "min = " << min << '\n';
21
22     return 0;
}
```

```
23 }
```

样例输入

```
1 23 56 19 11 78
```

预期输出

```
1 min = 11
```

Line 10 的语句 `std::cin >> numbers[i]` 从输入中读取一个整数并存入 `numbers[i]`，也就是 `numbers` 数组的第 `i` 个元素。

如果数组大小也是由用户输入指定的，该如何处理？一个简单的方法就是声明一个足够大的数组：

min-array-input-unknown-size.cpp

```
1 // File: min-array-input-unknown-size.cpp
2
3 #include <iostream>
4
5 int main() {
6     constexpr int MAX_SIZE{10000};
7     int numbers[MAX_SIZE]{};
8
9     int size{};
10    std::cin >> size;
11
12    for (int i{0}; i < size; i++) {
13        std::cin >> numbers[i];
14    }
15
16    int min{numbers[0]};
17    for (int i{1}; i < size; i++) {
18        if (numbers[i] < min) {
19            min = numbers[i];
20        }
21    }
22
23    std::cout << "min = " << min << '\n';
24
25    return 0;
26 }
```

样例输入

```
1 5
2 23 56 19 11 78
```

预期输出

```
1 min = 11
```

! Line 7 的 `numbers` 数组是自动存储期 (automatic storage duration) 对象 (object)。在程序运行时, `numbers` 通常被分配在栈 (stack) 上, 而每个程序可用的栈空间相当有限。比如 Linux 操作系统可能默认设置栈空间最大为 8M。假设每个 `int` 占 4 字节, 则 `numbers` 的大小最多约为 200 万。如果需要声明超过此大小限制的数组, 则可以将 `numbers` 声明语句放在 `main` 函数之前, 此时 `numbers` 是静态存储期 (static storage duration) 对象, 被分配在特定区域的空间上。

关于变量的存储期, 后续课程会详细介绍。

2.4 for 语句与作用域

现在考虑 `min-array-input.cpp` 中两处 `for` 语句 (Line 9 与 Line 14)。它们的第一部分都包含 `int i{0}` (或 `int i{1}`) 这个声明语句。一般来说, 变量不可重定义 (re-defined)。这里两个 `i` 没有冲突, 是因为这两个变量 `i` 的作用域 (scope) 不同: 在 C++ 中, `for` 语句第一部分声明的变量, 其作用域被限制在该 `for` 循环中, 包括循环条件表达式、迭代表达式与循环体。这意味着, (1) 这两个变量 `i` 是不同的变量; (2) 在 `for` 循环之外, 无法访问这两个 `i` 变量。

如果把循环计数器声明在 `for` 语句之外, 作用域就会扩大到整个函数:

代码块

```
1 int i{};
2 for (i = 0; i < NUM; i++) {
3     // ...
4 }
5
6 int j{};
7 for (j = 0; j < NUM; j++) {
8     // ...
9 }
```

此时，`i` 与 `j` 的作用域都扩大到整个函数，必须为第二个循环另取名字（如 `j`），而且后续代码很可能不小心误用变量 `i` 与 `j`。这样的代码是有失优雅的，不符合良好的编程习惯。

! 很多遗产 (legacy) 代码，包括有的 C 标准库代码，可能因为遵循 C99 之前的标准，在现在看来很“丑陋”。我们没有资格“嘲笑”它们。

3. palindrome-for

判定给定的字符串是否为回文串。

回文串问题与 `while` 版本参见《3-while》第 6.2 节：用两个下标 `left` 与 `right` 从两端向中间逐对比较字符，发现某一对不相等就立即断定它不是回文串。

在 `while` 版本中，循环的三件事分散在三处：在循环之前声明 `left` 与 `right`，在 `while` 后的括号里写条件 `left < right`，在循环体末尾写 `++left` 与 `--right`。改用 `for` 后，这三部分被聚拢到 `for` 的括号里——init-clause 同时声明并初始化 `left`、`right`，cond-expression 仍是 `left < right`，iteration-expression 用逗号表达式同时完成 `++left` 与 `--right`：

palindrome-for.cpp

```
1 // File: palindrome-for.cpp
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     std::string s{};
8     std::cin >> s;
9
10    bool is_palindrome{true};
11    for (std::size_t left{0}, right{s.size() - 1}; left < right; ++left, --
right) {
12        if (s.at(left) != s.at(right)) {
13            is_palindrome = false;
14            break;
15        }
16    }
17
18    if (is_palindrome) {
19        std::cout << std::format("{} is a palindrome.\n", s);
20    } else {
21        std::cout << std::format("{} is not a palindrome.\n", s);
22    }
23 }
```

```
24     return 0;
25 }
```

样例输入 / 预期输出

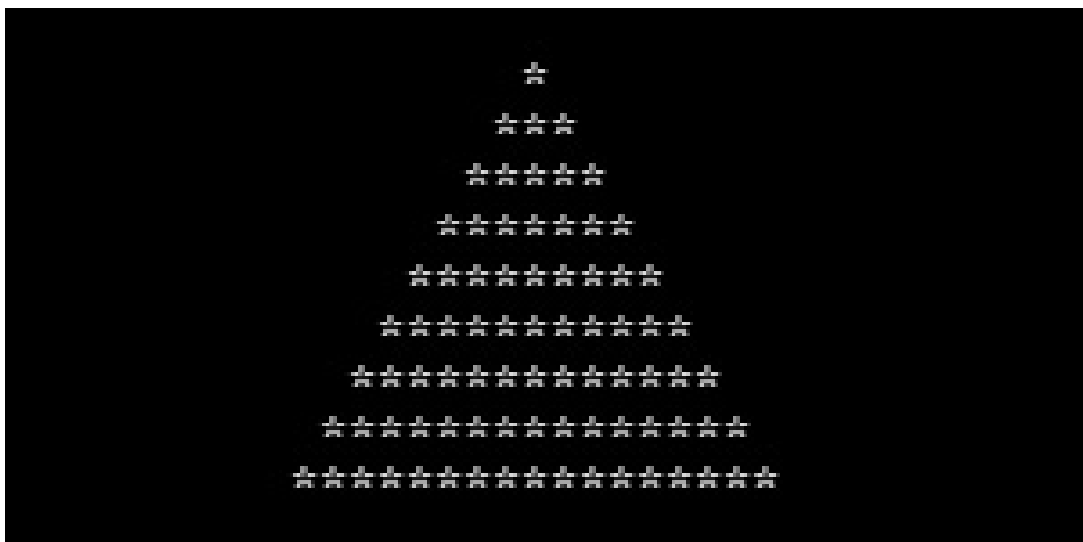
```
1  abcba =>  abcba is a palindrome.
2  abc   =>  abc is not a palindrome.
```

与 `while` 版本对比，这个例子展示了 `for` 语句在处理"带两个下标的对称遍历"时的优势：

- `init-clause` 可以在同一条声明里定义多个同类型变量（`std::size_t left{0}, right{...}`）；`iteration-expression` 可以用逗号表达式依次执行多个更新（`++left, --right`）。计数器的初始化与增量一目了然。
- 作用域更紧凑：`while` 版本的 `left`、`right` 声明在循环之外，循环结束后仍可见；`for` 版本把它们声明在 `init-clause` 中，作用域仅限于本次循环，循环结束后无法再访问，避免了与后续代码的命名冲突。
- 循环体只保留真正的比较逻辑（`if` 与 `break`），更聚焦于"做什么"，而不是"怎么挪下标"。

4. stars

打印指定行数（记为 `lines`）的星形金字塔，如下图所示。



我们需要两层嵌套的循环结构完成打印任务：

- 外层循环控制当前正在打印第几行；
- 内层循环控制如何打印当前行。

要打印当前某行，我们又需要两个并列的循环结构：

- 第一个循环用于打印若干个空格；
- 第二个循环用于打印若干个星形。

接下来的任务就简单了，只需要确定当前行（假设为第 i 行）需要打印的空格数（ $\text{lines} - 1 - i$ ）与星形数（ $2 * i + 1$ ）。

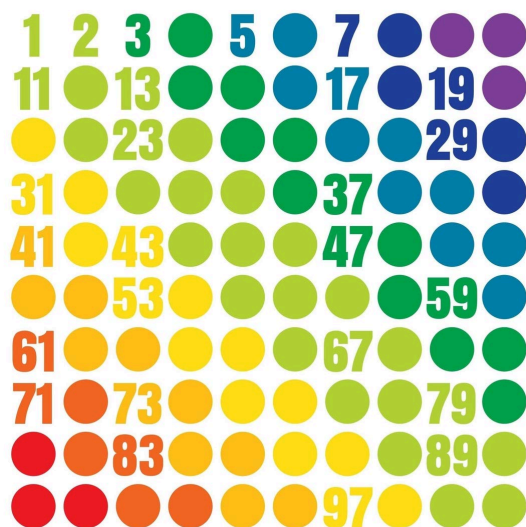
stars.cpp

```
1 // File: stars.cpp
2 #include <iostream>
3
4 int main() {
5     int lines{};
6     std::cin >> lines;
7
8     for (int i{0}; i < lines; i++) {
9         // Print (lines - 1 - i) spaces.
10        for (int j{0}; j < lines - 1 - i; j++) {
11            std::cout << ' ';
12        }
13        // Print (2 * i + 1) stars.
14        for (int j{0}; j < 2 * i + 1; j++) {
15            std::cout << '*';
16        }
17        std::cout << '\n';
18    }
19
20    return 0;
21 }
```

5. primes

输出从 1 到给定整数之间的所有素数。

PRIME NUMBERS



5.1 primes-bf

素数 (prime number) 是大于 1、且只能被 1 和自身整除的正整数。例如 2、3、5、7 都是素数，4、6、9 不是。

要输出 2 到 n 之间的所有素数，最直接的想法是：用一个循环逐个考察候选整数 i (i 从 2 取到 n)，判断 i 是否为素数，若是则打印。而"判断 i 是否为素数"本身又需要一个循环：从 2 开始逐个尝试整数 j ，看 i 能否被 j 整除。只要找到一个因子， i 就不是素数。

因此，这个问题需要两层嵌套的循环：

- 外层循环遍历候选数 i (从 2 到 n)；
- 内层循环对当前的 i 做试除 (trial division)，判断它是否为素数。

primes-bf.cpp

```
1 // File: primes-bf.cpp
2
3 #include <iostream>
4
5 int main() {
6     int n{};
7     std::cin >> n;
8
9     for (int i{2}; i <= n; i++) {
10         bool is_prime{true};
11         for (int j{2}; j < i; j++) {
12             if (i % j == 0) {
13                 is_prime = false;
14             }
15         }
```

```

16     if (is_prime) {
17         std::cout << i << ' ';
18     }
19 }
20
21 return 0;
22 }

```

`is_prime` 是一个 `bool` 类型的"标志变量" (flag variable)，先假设为真 (Line 10)，一旦在试除中发现因子就置为假 (Line 13)。内层循环结束后，根据 `is_prime` 的值决定是否打印 `i`。这是一种常见模式：先用一个布尔变量记录初步结论，再在循环中根据反例修正它。

`primes-bf.cpp` 有一个明显的性能缺陷：即使已经发现 `i` 有因子（比如 `i = 100`，`j = 2` 时就已经确定它不是素数），内层循环仍然要继续试到 `i - 1`，白白浪费时间。

5.2 primes-break

我们希望一旦确认 `i` 不是素数，就立即退出内层循环。这是 `break` 语句的典型应用场景。需要注意的是，在多层循环中，`break` 仅用于终止当前所在的内层循环。外层循环会继续考察下一个候选数 `i`，这正是我们想要的。

primes-break.cpp

```

1  // File: primes-break.cpp
2
3  #include <iostream>
4
5  int main() {
6      int n{};
7      std::cin >> n;
8
9      for (int i{2}; i <= n; i++) {
10         bool is_prime{true};
11         for (int j{2}; j < i; j++) {
12             if (i % j == 0) {
13                 is_prime = false;
14                 break; // a factor is found; stop dividing
15             }
16         }
17         if (is_prime) {
18             std::cout << i << ' ';
19         }
20     }
21
22     return 0;

```

```
23 }
```

加入 `break` 后，对于合数 `i`，内层循环通常很快结束；但对于素数 `i`，仍然要把 `2 .. i - 1` 全部试一遍。

5.3 primes-sqrt

内层循环真的需要试到 `i - 1` 吗？如果 `i = a × b`，那么 `a` 与 `b` 中必有一个不超过 $\sqrt{i}$ 。因此，内层循环只需试到 $\sqrt{i}$ 就足够了。

primes-sqrt.cpp

```
1 // File: primes-sqrt.cpp
2
3 #include <iostream>
4
5 int main() {
6     int n{};
7     std::cin >> n;
8
9     for (int i{2}; i <= n; i++) {
10         bool is_prime{true};
11         for (int j{2}; j * j <= i; j++) { // only need to try up to sqrt(i)
12             if (i % j == 0) {
13                 is_prime = false;
14                 break;
15             }
16         }
17         if (is_prime) {
18             std::cout << i << ' ';
19         }
20     }
21
22     return 0;
23 }
```

`j * j` (Line 11) 有可能超出 `int` 的表示范围。例如 `i = 2147483647` (即 `INT_MAX`，它本身是素数) 时，内层会一直试到 `j = 46341`，此时 `j * j = 463412 = 2147488281`，已经超出 `int` 的上限。也就是说，`j * j` 产生了溢出 (overflow)。然而，有符号整数溢出是未定义行为 (undefined behavior, UB)。

更稳妥的写法是用除法：`j <= i / j`，除法不会溢出。

 [Arithmetic operators@cppreference](#): When signed integer arithmetic operation overflows (the result does not fit in the result type), the behavior is undefined. (有符号整数溢出是未定义行为。)

5.4 primes

综合以上分析，得到完整程序：

```
primes.cpp
1  // File: primes.cpp
2
3  #include <iostream>
4
5  int main() {
6      int n{};
7      std::cin >> n;
8
9      for (int i{2}; i <= n; i++) {
10         bool is_prime{true};
11         for (int j{2}; j <= i / j; j++) { // avoid overflow: use division
12             if (i % j == 0) {
13                 is_prime = false;
14                 break;
15             }
16         }
17         if (is_prime) {
18             std::cout << i << ' ';
19         }
20     }
21     std::cout << '\n';
22
23     return 0;
24 }
```

样例输入 / 预期输出

```
1  30 => 2 3 5 7 11 13 17 19 23 29
```

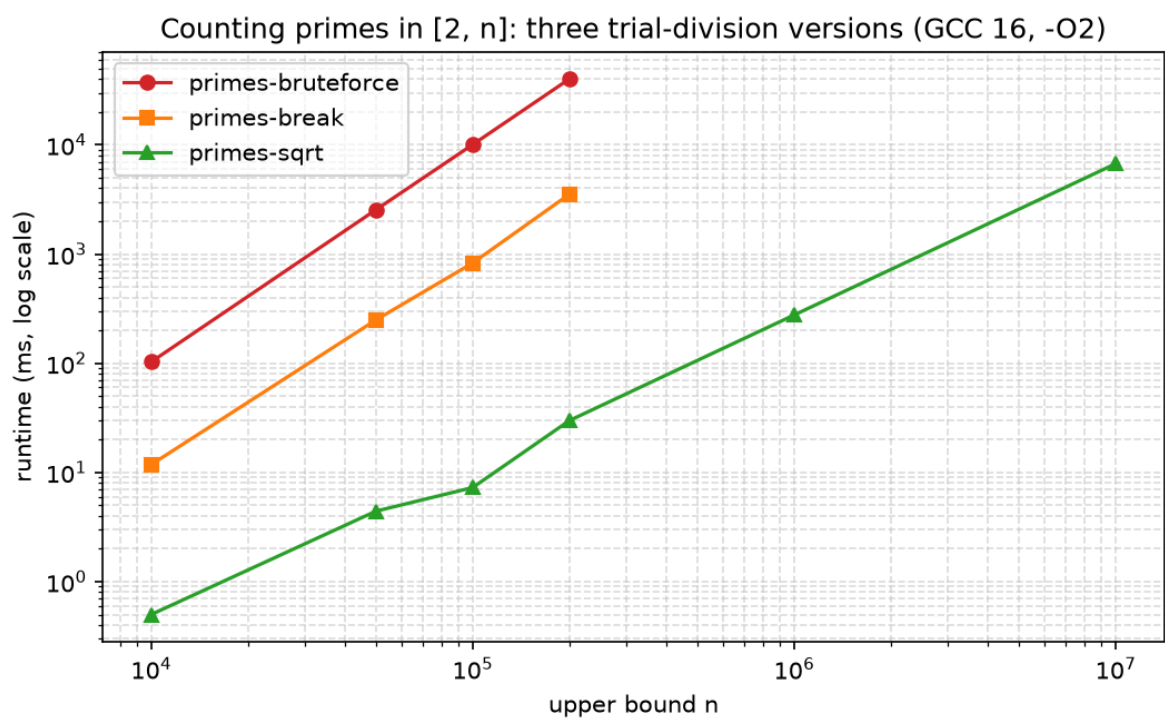
5.5 性能分析

`primes-bf`、`primes-break`、`primes` 三个版本的外层循环都枚举 `i` 从 2 到 `n`，区别在于内层试除循环的试除范围与退出时机：`primes-bf` 对每个 `i` 都试遍 `[2, i)`，发现因子后仍继

续试除；primes-break 同样试遍 [2, i)，但发现因子立即退出；primes-sqrt 只需试到 $\sqrt{i}$ 。

本节对三个版本的程序进行性能分析。为了避免逐个打印素数（std::cout）成为瓶颈，基准程序只在计数器上加一，循环结束后输出素数个数。

下面用 GCC 16（-O2）实测不同上界 n 的运行时间；纵轴为运行时间，采用对数坐标。



统计 [2, n] 内素数个数：三种试除版本的运行时间对比（GCC 16, -O2，纵轴对数）

实测数据如下（本地机器、GCC 16、-O2；"—"表示该版本在合理时间内无法完成）：

上界 n	素数个数	primes-bf	primes-break	primes
10,000	1,229	103 ms	11.8 ms	0.50 ms
50,000	5,133	2,544 ms	249 ms	4.4 ms
100,000	9,592	10.1 s	828 ms	7.3 ms
200,000	17,984	40.2 s	3.5 s	29.9 ms
1,000,000	78,498	—	—	277 ms
10,000,000	664,579	—	—	6.72 s



 primes-bench.cpp

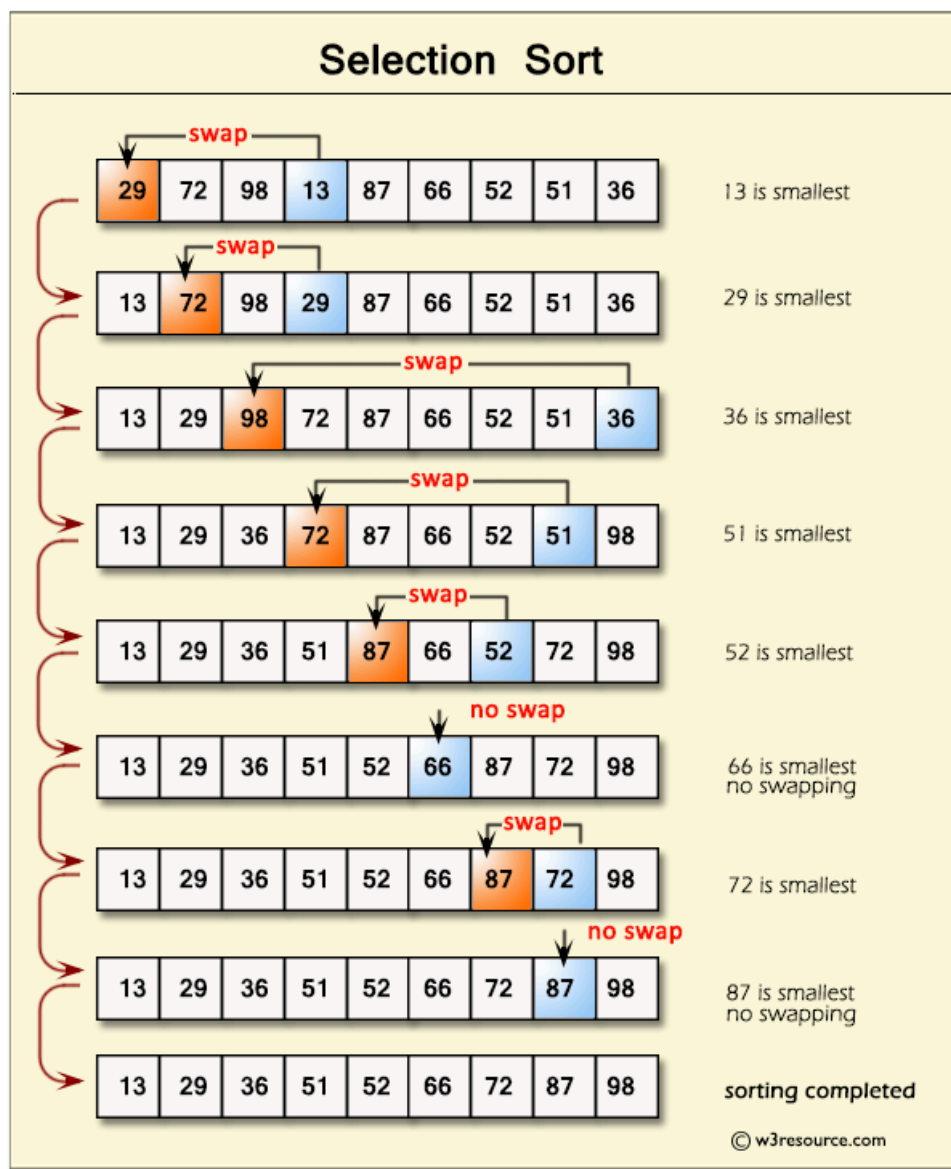
6. selection-sort

6.1 排序算法

[排序 \(Sorting\)](#) 是计算机科学中最基础的算法问题之一。顾名思义，给定一组数据（以整数为例），排序算法要求对其进行排序。排序算法多种多样，其中较为经典的，也不下十余种，包括选择排序 ([Selection sort](#))、插入排序 ([Insertion sort](#))、冒泡排序 ([Bubble sort](#))、合并排序 ([Merge sort](#))、快速排序 ([Quicksort](#))、堆排序 ([Heapsort](#))、希尔排序 ([Shellsort](#))、桶排序 ([Bucket sort](#))、计数排序 ([Counting sort](#))、基数排序 ([LSD Radix Sort](#)) 等。本节介绍易于理解的选择排序，其目的仍是学习如何运用循环结构。下节将介绍合并排序与插入排序。

6.2 选择排序

选择排序的核心在于“选择”。以下图为例，输入的整数数组大小 9。



- 在第 0 次迭代时，算法选择整个数组 $[0 \dots 8]$ 中的最小值 13，将其与下标为 0 的元素（29）交换位置。此时，下标为 0 的位置已经填入了正确的元素。因此，问题转化为“对剩余数组 $[1 \dots 8]$ 进行排序”。该问题与原问题本质相同，只不过在规模上少了一个元素。
- 在第 1 次迭代时，算法选择数组 $[1 \dots 8]$ 中的最小值 29，将其与下标为 1 的元素（72）交换位置。此时，下标为 1 的位置也填入了正确的元素。因此，问题转化为“对剩余数组 $[2 \dots 8]$ 进行排序”。该问题也与原问题本质相同，只不过在规模上又少了一个元素。
- 重复以上过程。一般地，在第 i 次迭代时，算法选择数组 $[i \dots 8]$ 中的最小值 min ，将其与下标为 i 的元素交换位置。此时，下标为 $0 \dots i$ 的位置均填入了正确的元素。因此，问题转化为“对剩余数组 $[i + 1 \dots 8]$ 进行排序”。

6.2.1 识别循环结构

在上述过程中，我们不断在产生新的子问题：假设数组中包含 n 个元素，子问题便是对 $[i \dots n - 1]$ 进行排序，其中 i 取值从 0 到 $n - 1$ 。循环体要做的事情就是将当前的子问题 $[i \dots n - 1]$ 转化成更小的子问题 $[i + 1 \dots n - 1]$ 。子问题规模不断缩小，直到 $i > n - 1$ 时，循环结束。

6.2.2 selection-sort

selection-sort.cpp

```
1  // File: selection-sort.cpp
2  #include <iostream>
3
4  int main() {
5      constexpr int LEN{20};
6      int numbers[LEN]{};
7
8      int len{0};
9      while (len < LEN && std::cin >> numbers[len]) {
10         ++len;
11     }
12
13     for (int i{0}; i < len; i++) {
14         std::cout << numbers[i] << ' ';
15     }
16     std::cout << '\n';
17
18     // Selection sort.
19     for (int i{0}; i < len; i++) {
20         // numbers[0 .. i-1] contains the correct elements.
21         // Find the minimum of numbers[i .. len-1].
22         int min{numbers[i]};
23         int min_index{i};
24
25         for (int j{i + 1}; j < len; j++) {
26             if (numbers[j] < min) {
27                 min = numbers[j];
28                 min_index = j;
29             }
30         }
31
32         // Swap numbers[i] and numbers[min_index].
33         int temp{numbers[i]};
34         numbers[i] = numbers[min_index];
35         numbers[min_index] = temp;
36
37         // Now numbers[0 .. i] contains the correct elements.
38     }
39     // Now numbers[0 .. len-1] is sorted.
40
41     for (int i{0}; i < len; i++) {
42         std::cout << numbers[i] << ' ';
43     }
```

```

44     std::cout << '\n';
45
46     return 0;
47 }

```

- Line 9 的 `while` 循环不断接受用户输入的整数，将其保存到数组 `numbers` 中；循环结束后，`len` 记录了数组最后一个元素的下标。我们在第 6.4 节“不定数目输入”中介绍此处 `while` 循环的含义。
- 在第 `i` 迭代开始的时候，`numbers[0 .. i - 1]` 已经包含正确的元素（Line 20）。第 `i` 迭代的任务是确定 `numbers[i]` 位置的元素，使得在此次迭代结束后，`numbers[0 .. i]` 均包含正确的元素（Line 37）。当所有迭代均结束时，`numbers[0 .. len - 1]` 就完成了排序（Line 39）。
- 外层循环体做两件事：第一，Lines 22--30 找出 `numbers[i .. len-1]` 中的最小值（记为 `min`）以及对应的下标（记为 `min_index`）；第二，Lines 32--35 交换 `numbers[i]` 与 `numbers[min_index]` 的值。

6.2.3 元素交换

要交换两个变量 `a` 与 `b` 的值，不能直接使用如下代码：

代码块

```

1  int a{5};
2  int b{10};
3
4  a = b; // a = 10, b = 10
5  b = a; // a = 10, b = 10

```

这会导致 `a = 10` 与 `b = 10`。究其原因，是因为 Line 4 赋值语句覆盖了 `a` 原本的值。因此，正确的做法应该是先保存一下 `a` 的值，如下所示：

代码块

```

1  int a{5};
2  int b{10};
3
4  int temp{a}; // temp = 5
5  a = b;       // a = 10
6  b = temp;    // b = 5

```

该方法需要引入一个临时变量。下面的方法利用位运算的性质，不需要临时变量，供参考：

```

1 int a{5};
2 int b{10};
3
4 a = a ^ b;
5 b = a ^ b; // b = (a ^ b) ^ b = a ^ (b ^ b) = a
6 a = a ^ b; // a = (a ^ b) ^ a = (a ^ a) ^ b = b

```

6.2.4 算法复杂度

在最坏情况下（输入数组按逆序排列时），对于大小为 n 的数组，选择排序需要进行约 n^2 次元素比较操作。我们称其算法复杂度为平方级别（quadratic）的。

6.3 扩展：其它排序算法

选择排序、插入排序与冒泡排序都是比较直观的排序算法，它们的最坏情况时间复杂度均为平方级别的。合并排序、快速排序、堆排序的性能更好，可以将时间降低到 $n \log n$ 级别。这些算法中的关键操作都是对元素进行比较。另外还有一类不基于比较的排序算法，比如桶排序、计数排序与基数排序，它们可以做到线性的时间复杂度。最后，值得一提的是，基于比较的排序算法的最优时间复杂度即为 $n \log n$ 。你能尝试给出证明吗？

6.4 不定数目输入

在 C++ 中，`std::cin >> x` 读取成功时返回 `cin`；而 `cin` 置于布尔上下文中时，一旦到达输入末尾（EOF）或读取出错就变为 `false`。因此不定数目输入可以写成：

不定数目输入（C++）

```

1 int len{0};
2 while (len < MAX_SIZE && std::cin >> numbers[len]) {
3     ++len;
4 }

```

运行时在新一行按 `Ctrl + D`（Linux/macOS）或 `Ctrl + Z`（Windows）表示输入结束。循环结束后，`len` 为成功读取的元素个数。

7. 修订记录

- 2026-09-17：统一代码块 caption 格式——去掉文件名后的（C++20）标记（如 `.cpp`（C++20） $\rightarrow$ `.cpp`）。
- 2026-09-18：新增第三、四、五章（stars.c、primes.c、selection-sort.c），内容自《3-while》移入；两处 Bilibili 视频以链接卡片形式保留。

- 2026-09-23: 将《3-while【Being Updated】》第4节 `prime` (含4.5「性能分析」及附图与基准脚本 `prime_bench.cpp`) 移入本文档末尾备用, 作为第6章; 修订记录顺延为第7章。
- 2026-09-27: 将原第6章 `prime` (判断单个整数是否为素数) 整合进第4章 `primes.c` ——从"输出 1..n 所有素数"的双重循环切入 (外层枚举候选数、内层试除判定素数), 循序给出 `primes-bruteforce`、`primes-break`、`primes-sqrt` 三个版本及最终 `primes.cpp`; 原第6章删除, 性能分析 (含基准图、表与 `prime_bench.cpp`) 并入4.5节。
- 2026-09-27: 将第1.1节 `array` (C传统数组的声明、初始化、下标访问与 `sizeof` 求大小) 的详细内容移至《3-while》第7.1节整合; 本节仅保留指向3-while 7.1的交叉引用。
- 2026-09-27: 补全1.2 `min-array-while` ——将问题引入 (min 迭代公式与 Sum 图) 从1.3移入本节, 给出 `min-array-while.cpp` (while 版本), 指出 while 把初始化/条件/增量分散在三处、易漏 `i++` 导致死循环, 由此引出1.3的 for 版本; 1.3 开头过渡句相应调整。
- 2026-09-27: 新增第3章 `palindrome-for` (位于 `min-array-input` 之后) ——将《3-while》第6.2节 `palindrome.cpp` 的 while 版本改写为 for 版本 (init-clause 同时声明 `left/right`, iteration-expression 用逗号表达式 `++left, --right`), 并对比 while 与 for 在变量作用域、循环体聚焦度上的差异; `stars.c` 及后续章节自动顺延。
- 2026-09-27: 重写4.5「性能分析」——度量对象由"判定单个整数是否为素数"改为"统计 [2, n] 内素数个数"。基准程序只计数、循环结束后才输出个数, 避免打印成为瓶颈; 重新实测三版 (GCC 16、-O2) 在 $n=10^4$ 到 10^7 下的耗时并绘制 log-log 对比图, 数据表同步更新, 附随源码替换为 `primes-bench.cpp`。
- 2026-09-27: 新增第1章「简介」 (学习目标/重点/难点, 仿《3-while》体例), 其余章节自动顺延; 将5.4「不定数目输入」与5.2.2「selection-sort」由C版本 (`stdio.h` / `scanf` / `printf` / `#define`) 改写为C++版本 (`<iostream>`、`std::cin` / `std::cout`、`constexpr`、统一初始化 `{}`), 并同步更新行号引用。
- 2026-09-27: 按 AI Writing Guidelines 统一 C++ 风格——`stars` 章代码由C改写为C++ (`<iostream>`、`std::cin` / `std::cout`、统一初始化 `{}`、删 TODO 占位注释、2 空格缩进); 三章标题去掉 `.c` 后缀 (`stars` / `primes` / `selection-sort`); 变量命名统一 snake_case (`is_palindrome`、`min_index`); 6.2.3 元素交换三段C片段改C++ 统一初始化并修正异或片段中的中文全角分号; `primes-sqrt` 的 `cppreference` 链接补全缺失的 `/w/` 路径; 性能分析正文 `primes-bruteforce` 统一为 `primes-bf`; `selection-sort` 行号引用按当前代码校正。

(注: 部分内容可能由 AI 生成)